MongoDB Atlas Setup Guide for GEMA

Complete guide to setting up MongoDB Atlas for your GEMA production deployment.

Table of Contents

---

1. Create MongoDB Atlas Account

Step 1.1: Sign Up

1. Go to MongoDB Atlas (https://www.mongodb.com/cloud/atlas)
2. Click **"Start Free"** or **"Try Free"**
3. Sign up with email or Google/GitHub account
4. Verify your email address

Step 1.2: Create Organization

1. After login, click **"Create an Organization"**
2. Organization Name: `GEMA Production` (or your preferred name)
3. Click **"Next"**
4. Add team members (optional)
5. Click **"Create Organization"**

---

2. Create a Cluster

Step 2.1: Create Project

1. Click **"New Project"**
2. Project Name: `GEMA` or `GEMA-Production`
3. Click **"Next"** → **"Create Project"**

Step 2.2: Build a Cluster

1. Click **"Build a Database"**
2. Choose deployment type:
    - **Shared (Free)**: M0 - 512 MB storage (Good for staging/testing)
    - **Dedicated**: M10+ (Recommended for production)

For Development/Staging (Free Tier - M0):

- **Provider**: AWS, Google Cloud, or Azure
- **Region**: Choose closest to your Hostinger server
    - If Hostinger is in EU: Select EU region
    - If Hostinger is in US: Select US region
    - If Hostinger is in Asia: Select Asia region
- **Cluster Name**: `gema-cluster` or `gema-dev`
- Click **"Create"**

For Production (Recommended - M10):

- **Provider**: AWS (recommended for stability)
- **Region**: Same as above (close to your server)
- **Cluster Tier**: M10 (2GB RAM, 10GB storage)
    - Cost: ~$57/month
    - Can handle moderate traffic
- **Additional Settings**:
    - **MongoDB Version**: 7.0 (latest)
    - **Backup**: Enable continuous backups (if available)
- **Cluster Name**: `gema-production`
- Click **"Create"**

> ⚠ **Important**: Cluster creation takes 5-10 minutes

---

3. Configure Network Access

Step 3.1: Add IP Whitelist

1. Go to **"Network Access"** (left sidebar)
2. Click **"Add IP Address"**

Option A: Allow Access from Anywhere (Development Only)

- Click **"Allow Access from Anywhere"**
- IP Address: `0.0.0.0/0`
- ⚠ **WARNING**: Only use this for development/testing!

Option B: Specific IP (Production - Recommended)

- Get your Hostinger server IP:

```
curl ifconfig.me
```

- Add IP Address: `YOUR_SERVER_IP/32`
- Description: `Hostinger KVM1 Production Server`

Option C: Multiple IPs (Best Practice)

- Add Hostinger server IP
- Add your development machine IP
- Add CI/CD server IP (if applicable)

3. Click **"Confirm"**

Step 3.2: Enable VPC Peering (Optional - Advanced)

For enhanced security in production, consider VPC peering if using AWS/GCP/Azure.

---

4. Create Database User

Step 4.1: Create User

1. Go to **"Database Access"** (left sidebar)
2. Click **"Add New Database User"**

Step 4.2: Configure User

Authentication Method: Password

User Credentials:

- **Username**: `gema-app` (or your preferred username)
- **Password**: Click **"Autogenerate Secure Password"**
    - ⚠ **IMPORTANT**: Copy and save this password immediately!

- Store securely (password manager recommended)

Database User Privileges:

- Select: **"Read and write to any database"**

Temporary User (Optional):

- Leave unchecked for permanent access

3. Click **"Add User"**

> **Best Practice**: Use different users for production and development environments

---

5. Get Connection String

Step 5.1: Connect to Cluster

1. Go to **"Database"** (left sidebar)
2. Find your cluster (e.g., `gema-production`)
3. Click **"Connect"**

Step 5.2: Choose Connection Method

1. Select **"Connect your application"**
2. **Driver**: Node.js
3. **Version**: 5.5 or later (matches your backend)

Step 5.3: Copy Connection String

You'll see something like:

```
mongodb+srv://gema-app:<password>@gema-production.xxxxx.mongodb.net/?retryWrites=true&w=majority
```

Modify the connection string:

```
# Original
mongodb+srv://gema-app:<password>@gema-production.xxxxx.mongodb.net/?retryWrites=true&w=majority

# Replace <password> with actual password
# Add database name before the query string

# Final format for GEMA:
mongodb+srv://gema-app:YOUR_ACTUAL_PASSWORD@gema-production.xxxxx.mongodb.net/gema?
retryWrites=true&w=majority
```

Step 5.4: Update Backend .env

Add to `/var/www/gema/backend/.env`:

```
MONGODB_URI=mongodb+srv://gema-app:YOUR_ACTUAL_PASSWORD@gema-production.xxxxx.mongodb.net/gema?
retryWrites=true&w=majority
```

Additional MongoDB Configuration (Optional):

```
# Connection pool size
MONGODB_POOL_SIZE=10

# Connection timeout
MONGODB_CONNECT_TIMEOUT=30000

# Server selection timeout
MONGODB_SERVER_TIMEOUT=30000
```

---

6. Create Database and Collections

MongoDB will auto-create the database and collections when your app first connects, but you can pre-create them:

Step 6.1: Open MongoDB Shell

1. Go to your cluster
2. Click **"Connect"**
3. Select **"MongoDB Shell"**
4. Follow instructions to connect

Or use MongoDB Compass (GUI):

1. Click **"Connect"**
2. Select **"Compass"**
3. Download and install MongoDB Compass
4. Use connection string to connect

Step 6.2: Create Collections (Optional)

Your backend will auto-create these, but for reference:

```
// Collections that GEMA will create:
- users
- events
- tickets
- orders
- blogs
- blogcategories
- blogcomments
- coupons
- vendors
- reviews
- notifications
- sessions
- uploads
- settings
```

---

7. Configure Indexes for Performance

Why Indexes Matter

Indexes dramatically improve query performance for production workloads.

Step 7.1: Access Cluster

1. Go to **"Database"** → **"Browse Collections"**
2. Select `gema` database

Step 7.2: Create Indexes

Click on each collection and create indexes:

Users Collection

```
// Email index (unique)
{ email: 1 }, { unique: true }

// Role index
{ role: 1 }

// Created at index
{ createdAt: -1 }
```

Events Collection

```
// Status and date compound index
{ status: 1, startDate: -1 }

// Vendor index
{ vendorId: 1 }

// Category index
{ category: 1 }

// Text search index for title and description
{ title: "text", description: "text" }

// Location index (for geospatial queries if needed)
{ "location.coordinates": "2dsphere" }
```

Orders Collection

```
// User and status compound index
{ userId: 1, status: 1 }

// Event index
{ eventId: 1 }

// Payment status index
{ paymentStatus: 1 }

// Created at index (for sorting)
{ createdAt: -1 }

// Order number index (unique)
{ orderNumber: 1 }, { unique: true }
```

Tickets Collection

```
// Event index
{ eventId: 1 }

// Order index
{ orderId: 1 }

// QR code index (unique)
{ qrCode: 1 }, { unique: true }

// Status index
{ status: 1 }
```

Blogs Collection

```
// Published and date compound index
{ published: 1, publishedAt: -1 }

// Category index
{ category: 1 }

// Author index
{ authorId: 1 }

// Slug index (unique)
{ slug: 1 }, { unique: true }

// Text search index
{ title: "text", content: "text" }
```

Coupons Collection

```
// Code index (unique)
{ code: 1 }, { unique: true }

// Valid dates compound index
{ validFrom: 1, validUntil: 1 }

// Active status index
{ active: 1 }
```

Step 7.3: Create Indexes via MongoDB Compass

1. Open MongoDB Compass
2. Connect using your connection string
3. Navigate to collection
4. Click **"Indexes"** tab
5. Click **"Create Index"**
6. Add index fields and options
7. Click **"Create Index"**

Step 7.4: Create Indexes via MongoDB Shell

```
use gema;

// Users
db.users.createIndex({ email: 1 }, { unique: true });
db.users.createIndex({ role: 1 });
db.users.createIndex({ createdAt: -1 });

// Events
db.events.createIndex({ status: 1, startDate: -1 });
db.events.createIndex({ vendorId: 1 });
db.events.createIndex({ category: 1 });
db.events.createIndex({ title: "text", description: "text" });

// Orders
db.orders.createIndex({ userId: 1, status: 1 });
db.orders.createIndex({ eventId: 1 });
db.orders.createIndex({ paymentStatus: 1 });
db.orders.createIndex({ createdAt: -1 });
db.orders.createIndex({ orderNumber: 1 }, { unique: true });

// Tickets
db.tickets.createIndex({ eventId: 1 });
db.tickets.createIndex({ orderId: 1 });
db.tickets.createIndex({ qrCode: 1 }, { unique: true });
db.tickets.createIndex({ status: 1 });

// Blogs
db.blogs.createIndex({ published: 1, publishedAt: -1 });
db.blogs.createIndex({ category: 1 });
db.blogs.createIndex({ authorId: 1 });
db.blogs.createIndex({ slug: 1 }, { unique: true });
db.blogs.createIndex({ title: "text", content: "text" });

// Coupons
db.coupons.createIndex({ code: 1 }, { unique: true });
db.coupons.createIndex({ validFrom: 1, validUntil: 1 });
db.coupons.createIndex({ active: 1 });
```

---

8. Setup Backups

Free Tier (M0)

- **Cloud Backups**: Not available
- **Manual Backups**: Use `mongodump` manually

Paid Tiers (M10+)

Step 8.1: Enable Cloud Backups

1. Go to **"Database"** → Select cluster
2. Click **"Backup"** tab
3. Click **"Enable Cloud Backups"**

Step 8.2: Configure Backup Policy

Snapshot Schedule:

- **Snapshot Frequency**: Every 12 hours (recommended)
- **Retention**: 7 days (adjust based on needs)

Point-in-Time Restores:

- Enable for mission-critical data
- Allows restore to any point within retention window

Step 8.3: Test Restore (Important!)

1. Go to **"Backup"** → **"Snapshots"**
2. Select a snapshot
3. Click **"Restore"**
4. Choose **"Download"** or **"Restore to Cluster"**

> **Best Practice**: Test restore procedure quarterly

---

9. Monitoring and Alerts

Step 9.1: Setup Alerts

1. Go to **"Alerts"** (left sidebar)
2. Click **"Add Alert"**

Recommended Alerts:

1. Connections Alert

- **Metric**: Connections
- **Threshold**: > 80% of max connections
- **Action**: Send email

2. Disk Space Alert

- **Metric**: Disk Space Used
- **Threshold**: > 75%
- **Action**: Send email

3. CPU Alert

- **Metric**: CPU Usage
- **Threshold**: > 80%
- **Action**: Send email

4. Replica Set Elections

- **Metric**: Replica Set Elections
- **Threshold**: > 0
- **Action**: Send email

Step 9.2: Performance Monitoring

1. Go to **"Metrics"** tab
2. Monitor:
    - **Query Performance**: Slow queries
    - **Index Usage**: Unused indexes
    - **Connection Count**: Connection pool health

- **Replication Lag**: Data sync status

Step 9.3: Real-Time Performance Panel

1. Go to **"Performance Advisor"**
2. Review recommendations for:
    - Missing indexes
    - Slow queries
    - Schema optimization

---

10. Security Best Practices

☑ DO:

1. **Use Strong Passwords**
    - Auto-generate passwords (16+ characters)
    - Store in password manager
2. **Restrict IP Access**
    - Whitelist only necessary IPs
    - Use VPC peering if possible
3. **Enable Encryption**
    - Encryption at rest (enabled by default on M10+)
    - TLS/SSL for connections (enabled by default)
4. **Separate Users**
    - Development user with limited access
    - Production user with necessary privileges
    - Admin user for management only
5. **Regular Audits**
    - Review access logs monthly
    - Check connected applications
    - Monitor unusual activity
6. **Use Environment Variables**
    - Never hardcode connection strings
    - Use .env files (not committed to git)

❌ DON'T:

1. **Never use `0.0.0.0/0` in production**
2. **Never commit connection strings to git**
3. **Never share database credentials**
4. **Never use simple passwords**
5. **Never ignore security alerts**

Quick Reference: Connection String Format

```
# Standard format
mongodb+srv://<username>:<password>@<cluster>.mongodb.net/<database>?<options>

# GEMA production example
mongodb+srv://gema-app:SecurePass123!@gema-production.abc12.mongodb.net/gema?retryWrites=true&w=majority

# With additional options
mongodb+srv://gema-app:SecurePass123!@gema-production.abc12.mongodb.net/gema?
retryWrites=true&w=majority&maxPoolSize=50&minPoolSize=10
```

Troubleshooting

Connection Issues

Error: "Authentication failed"

- Check username/password in connection string
- Verify user exists in Database Access

Error: "Server selection timed out"

- Check IP whitelist in Network Access
- Verify server can reach internet
- Check firewall rules

Error: "Connection pool closed"

- Restart backend application
- Check MongoDB Atlas status page

Performance Issues

Slow Queries

- Check Performance Advisor for missing indexes
- Review slow query logs
- Optimize queries in backend code

High CPU Usage

- Add indexes for frequent queries
- Consider upgrading cluster tier
- Review query patterns

Next Steps After Setup

1. ☑ Update backend `.env` with connection string
2. ☑ Restart backend: `pm2 restart gema-backend`
3. ☑ Verify connection: Check backend logs
4. ☑ Create indexes for production
5. ☑ Setup monitoring alerts
6. ☑ Test backup/restore procedure
7. ☑ Document credentials securely

Useful Links

- MongoDB Atlas Dashboard (https://cloud.mongodb.com)
- MongoDB Atlas Documentation (https://docs.atlas.mongodb.com/)
- MongoDB Compass Download (https://www.mongodb.com/try/download/compass)
- Connection String Format (https://docs.mongodb.com/manual/reference/connection-string/)
- MongoDB Atlas Status (https://status.mongodb.com/)

Last Updated: 2025-11-22
For: GEMA Production Deployment